

Trabajo Práctico 2

I. Introducción a Spring Data JPA

1. Spring Data JPA y su lugar en el ecosistema

Inciso 1

¿Qué es Spring Data JPA? ¿Qué problema resuelve respecto de usar Hibernate directamente? Describir dos situaciones del proyecto donde Spring Data JPA simplifica código que en la Práctica 1 requería implementación manual

Respuesta:

Spring Data JPA es una capa de **abstracción** sobre el ORM (Hibernate) que forma parte del ecosistema Spring. No es un ORM en sí mismo, sino un framework diseñado para simplificar significativamente la construcción de la capa de acceso a datos.

Problemas que resuelve respecto de Hibernate directo

El principal problema que resuelve es la eliminación del **código boilerplate** (código repetitivo y manual). Mientras que Hibernate directo requiere que el desarrollador gestione la lógica técnica de persistencia, Spring Data JPA ofrece las siguientes ventajas:

- **Implementación automática de repositorios:** Evita tener que escribir clases concretas para cada repositorio. El framework genera la implementación en tiempo de ejecución (runtime) a través de proxies dinámicos.
- **Gestión transparente de la Sesión:** Elimina la necesidad de inyectar la `SessionFactory` y llamar manualmente a `getCurrentSession()` en cada método del repositorio.
- **Abstracción de operaciones comunes:** Provee automáticamente operaciones estándar de persistencia (CRUD) y capacidades avanzadas como paginación y ordenamiento nativo sin código adicional.

Situaciones del proyecto donde simplifica el código

Al migrar la Práctica 1 (Hibernate directo) a la Práctica 2 (Spring Data JPA), se identifican dos situaciones clave de simplificación:

1. Definición de Repositorios mediante Interfaces:

- **En la Práctica 1:** Se debía implementar una clase concreta por cada repositorio (ej. `PurchaseRepositoryImpl`), inyectar la `SessionFactory`, obtener la sesión activa y codificar manualmente métodos como `save()`, `buscarPorId()` o `eliminar()`.
- **Con Spring Data JPA:** El repositorio pasa a ser una **interfaz** que extiende de `CrudRepository<Entidad, ID>`. Spring Data JPA implementa automáticamente todos los métodos básicos de persistencia, eliminando decenas de líneas de código manual.

2. Generación de Consultas mediante Query Methods:

- **En la Práctica 1:** Para realizar una consulta simple (como buscar un usuario por su email), era necesario escribir manualmente la consulta en **HQL**, crear el objeto `Query`, setear los parámetros y ejecutar la sesión.
- **Con Spring Data JPA:** Se utilizan los **Query Methods**, donde el framework deriva la consulta SQL automáticamente a partir del nombre del método definido en la interfaz (ej. `findByEmail(String email)`). Esto mejora la legibilidad y reduce el acoplamiento a la infraestructura de persistencia.

Inciso 2

Spring Data JPA no es un ORM sino una capa de abstracción sobre el ORM. Explicar la diferencia: ¿qué hace Spring Data JPA? ¿Qué sigue haciendo Hibernate internamente?

Respuesta:

Spring Data JPA no es un motor de persistencia (ORM) en sí mismo, sino una **capa de abstracción** diseñada para simplificar el uso de JPA y Hibernate, eliminando la necesidad de escribir código repetitivo o "boilerplate".

A continuación se detalla la división de responsabilidades entre ambas tecnologías:

¿Qué hace Spring Data JPA?

Su función principal es facilitar la construcción de la capa de acceso a datos (Repositorios). Entre sus tareas específicas se encuentran:

- **Generación de Repositorios en Runtime:** El desarrollador solo define **interfaces** (que extienden de `CrudRepository` o `JpaRepository`) y el framework genera la implementación concreta automáticamente en tiempo de ejecución mediante **proxies dinámicos**.
- **Implementación Automática de CRUD:** Provee de forma nativa los métodos estándar como `save()`, `findById()`, `findAll()` y `deleteById()`, evitando que el programador tenga que codificarlos manualmente con la `Session` de Hibernate.
- **Query Methods (Consultas por nombre):** Es capaz de **derivar consultas SQL** automáticamente a partir del nombre de un método definido en la interfaz (por ejemplo, `findByEmail`), ahorrando la escritura de HQL o JPQL para operaciones simples.
- **Gestión Declarativa de Transacciones:** Permite propagar y controlar los límites de la unidad de trabajo mediante la anotación `@Transactional`.
- **Soporte de Paginación y Ordenamiento:** Ofrece abstracciones como `Pageable` y `Sort` para manejar grandes volúmenes de datos de forma nativa.

¿Qué sigue haciendo Hibernate internamente?

Hibernate continúa siendo el **motor de persistencia (ORM)** que realiza el "trabajo sucio" de bajo nivel. Aunque Spring Data JPA lo "envuelve", Hibernate sigue siendo responsable de:

- **Gestionar el Contexto de Persistencia:** Mantiene el control del **ciclo de vida de las entidades** (estados *managed*, *detached*, *removed*, etc.) y asegura que los cambios en los objetos se sincronicen con la base de datos.
- **Mapeo Objeto-Relacional:** Es el encargado de interpretar las anotaciones (`@Entity`, `@Table`, `@OneToMany`) para saber cómo transformar una clase en una tabla y viceversa.
- **Traducción a SQL:** Spring Data JPA le entrega una consulta en JPQL o un criterio de búsqueda, y Hibernate es quien la **traduce al dialecto SQL específico** del motor de base de datos utilizado (Oracle, MySQL, etc.).
- **Mapeo de ResultSet:** Transforma las filas y columnas devueltas por la base de datos en **objetos Java "reales"**.

- **Gestión de Proxies y Lazy Loading:** Controla cuándo recuperar un colaborador de una entidad (carga diferida) para optimizar el rendimiento.

En resumen, mientras **Spring Data JPA** se encarga de que escribas **menos código** para definir cómo acceder a los datos, **Hibernate** se encarga de la **lógica técnica** de cómo esos datos se mueven entre el mundo de los objetos y el mundo relacional.

Inciso 3

La siguiente tabla lista tareas relacionadas con la persistencia. Marcar con una X la columna de la tecnología que resuelve ese problema en la nueva implementación con Spring Data JPA:

Tarea	JDBC	Hibernate	Spring Data JPA	Justificacion
Abrir y cerrar la conexión a la base de datos		X		En JDBC puro , el desarrollador es el responsable directo de abrir y cerrar las conexiones físicas mediante código manual. Hibernate y Spring Data JPA resuelven esto automatizándolo a través de la sesión y el pool.
Implementar save(), findById() y deleteById()			X	Spring Data JPA provee automáticamente la implementación de estas operaciones CRUD estándar, eliminando el código manual ("boilerplate") que se requería en la Práctica 1.

Tarea	JDBC	Spring Data Hibernate JPA	Justificacion
Gestionar el ciclo de vida de las entidades (@Entity)		X	Aunque Spring Data JPA lo abstraee, es Hibernate el que sigue siendo responsable internamente de gestionar el Contexto de Persistencia y los estados de las entidades (<i>managed, detached, removed</i>).
Derivar una consulta a partir del nombre del método		X	Esta es una funcionalidad exclusiva de Spring Data JPA llamada Query Methods , donde el framework interpreta nombres como <code>findByEmail</code> para generar la consulta.
Manejar el pool de conexiones		X	Hibernate se encarga de administrar el pool de conexiones a la base de datos para optimizar este recurso costoso, a menudo integrándose con librerías externas.
Propagar transacciones con @Transactional		X	La gestión declarativa de los límites de la unidad de trabajo mediante la anotación @Transactional es una capacidad proporcionada por el ecosistema de Spring.
Generar la implementación		X	Spring Data JPA genera las implementaciones concretas de tus

Tarea	JDBC	Hibernate	Spring Data JPA	Justificacion
del repositorio en runtime				interfaces de repositorio en tiempo de ejecución utilizando proxies dinámicos .
Mapear ResultSet a objetos Java		X		El "trabajo sucio" de transformar las filas y columnas del SQL en objetos Java reales lo sigue realizando Hibernate internamente.
Proveer soporte nativo de paginación (Pageable)			X	Spring Data JPA incorpora soporte nativo para paginar y ordenar resultados mediante las abstracciones Pageable y Sort .

2. Configuración del proyecto

Inciso 4

Listar los cambios que deben realizarse en el proyecto de la Práctica 1 para incorporar Spring Data JPA.

Respuesta:

1. Cambios en la Configuración e Infraestructura

- **Gestión de Dependencias:** Se deben actualizar las dependencias en el archivo **pom.xml** para incluir los starters de Spring Data JPA.
- **Reemplazo de la Configuración de Persistencia:** La clase **HibernateConfiguration** (usada en la Práctica 1 para obtener la **SessionFactory**) debe ser reemplazada por la clase **SpringDataConfiguration**.
- **Externalización de Propiedades:** La configuración técnica de la base de datos (URL, credenciales, dialecto y estrategia de generación de tablas) se

traslada de clases Java al archivo **application.properties** de Spring Boot.

- **Actualización de Clases de Apoyo:** Se deben modificar la clase `AppConfig`, el inicializador de la base de datos `DBInitializer` y los archivos de testing para que sean compatibles con el nuevo motor.

2. Rediseño de la Capa de Repositorios

- **De Clases a Interfaces:** Los repositorios ya no son clases concretas (implementaciones manuales), sino **interfaces**.
- **Herencia de CrudRepository:** Cada interfaz de repositorio (ej. `RouteRepository`) debe extender de `CrudRepository<Entidad, ID>`, especificando el tipo de entidad y su clave primaria.
- **Eliminación de Código "Boilerplate":** Se deben eliminar las implementaciones manuales de métodos como `save()`, `findById()` y `delete()`, ya que Spring Data JPA las genera automáticamente en tiempo de ejecución.

3. Migración de Consultas

- **Adopción de Query Methods:** Las consultas simples (como buscar por nombre o email) que antes requerían HQL manual ahora se resuelven mediante **métodos por nombre** definidos en la interfaz (ej. `findByUsername`).
- **Uso de @Query:** Para consultas complejas que involucren JOINS o agrupaciones, se debe utilizar la anotación `@Query` directamente sobre el método en la interfaz, eliminando la necesidad de manejar el objeto `Query` de Hibernate manualmente.

4. Adaptación de la Capa de Servicio

- **Inyección de Repositorios:** Se debe crear una nueva implementación del servicio (ej. `ToursServiceImpl`) que inyecte las **interfaces de los repositorios** en lugar de interactuar con la `SessionFactory`.
- **Actualización de Llamadas:** Las llamadas directas a la sesión de Hibernate (ej. `session.save` o `session.createQuery`) dentro de los servicios deben ser reemplazadas por los métodos equivalentes provistos por el nuevo repositorio.

Lo que NO debe cambiarse

- **Mapeo de Entidades:** El mapeo ya implementado (anotaciones `@Entity`, `@Table`, relaciones `@OneToMany` / `@ManyToMany`, estrategias de herencia, cascadas y `FetchType`) **no debe modificarse**, ya que Spring Data JPA utiliza la misma infraestructura de Hibernate para estas definiciones.

Inciso 5

En la Práctica 1 se configuraba Hibernate mediante una clase de configuración Java para obtener la SessionFactory. ¿Cómo se reemplaza esta configuración usando application.properties en Spring Boot?

1. Cambio de Paradigma: De Código a Propiedades

En la Práctica 1, se utilizaba una clase de configuración Java (usualmente llamada `HibernateConfiguration`) para instanciar manualmente la **SessionFactory** y definir los parámetros técnicos. En Spring Boot, esta configuración se externaliza al archivo `application.properties`, lo que permite que el framework gestione la infraestructura de persistencia de forma automática.

2. Mapeo de Propiedades Técnicas

Los valores que antes se codeaban en la clase Java ahora se definen mediante las siguientes propiedades estándar:

- **Datos de Conexión (Datasource):** Se configuran mediante `spring.datasource.url`, `spring.datasource.username`, `spring.datasource.password` y `spring.datasource.driver-class-name`.
- **Dialecto de Hibernate:** Se especifica a través de `spring.jpa.properties.hibernate.dialect`, indicando al ORM cómo comunicarse con el motor de base de datos específico (ej. MySQL).
- **Estrategia del Esquema:** La propiedad `spring.jpa.hibernate.ddl-auto` reemplaza la lógica manual para controlar si Hibernate debe **crear, actualizar o validar** las tablas de la base de datos automáticamente al iniciar la aplicación.

3. Reemplazo de la Clase de Configuración

A nivel de código, la clase `HibernateConfiguration` de la Práctica 1 es eliminada y reemplazada por la clase **SpringDataConfiguration**. Esta nueva clase trabaja en conjunto con las propiedades definidas en

`application.properties` para inicializar el contexto de Spring Data JPA sin necesidad de que el desarrollador gestione directamente la creación de la `SessionFactory`.

Inciso 6

Describir las propiedades más relevantes de Spring Data JPA que deben configurarse en `application.properties` para este proyecto. Incluir al menos: `datasource` (url, username, password, driver), dialecto de Hibernate, y la propiedad que controla si Hibernate debe crear, actualizar o validar el esquema. ¿Cuál de estos valores conviene usar durante el desarrollo?

Respuesta:

En la migración a **Spring Data JPA**, la configuración técnica de la persistencia se traslada al archivo **`application.properties`**, permitiendo que el framework gestione la infraestructura de forma automática y centralizada.

A continuación, se describen las propiedades más relevantes para este proyecto:

1. Configuración del Datasource (Conexión)

Estas propiedades definen cómo la aplicación se comunica físicamente con la base de datos:

- **`spring.datasource.url`**: Especifica la dirección de la base de datos (ej. `jdbc:mysql://localhost:3306/tours_db`).
- **`spring.datasource.username`**: El nombre de usuario para acceder al motor de base de datos.
- **`spring.datasource.password`**: La contraseña correspondiente al usuario.
- **`spring.datasource.driver-class-name`**: El nombre de la clase del driver JDBC específico del motor utilizado (ej. `com.mysql.cj.jdbc.Driver`).

2. Dialecto de Hibernate

- **`spring.jpa.properties.hibernate.dialect`**: Indica a Hibernate el protocolo SQL específico del motor que se está utilizando (ej.

`org.hibernate.dialect.MySQLDialect`). Esto es crucial para que el ORM traduzca correctamente las consultas HQL/JPQL al SQL nativo del motor.

3. Control del Esquema (DDL-Auto)

La propiedad **`spring.jpa.hibernate.ddl-auto`** controla si Hibernate debe intervenir en la estructura de las tablas al iniciar la aplicación. Los valores más comunes son:

- **update**: Actualiza el esquema existente añadiendo nuevas columnas o tablas sin borrar los datos previos.
- **create**: Crea las tablas desde cero cada vez que inicia la aplicación, borrando cualquier dato anterior.
- **create-drop**: Similar a *create*, pero además borra las tablas al cerrar la aplicación.
- **validate**: Solo verifica que el esquema de la base de datos coincida con el mapeo de las entidades, sin realizar cambios.
- **none**: No realiza ninguna acción sobre el esquema.

¿Cuál conviene usar durante el desarrollo?

Durante la etapa de **desarrollo**, lo más conveniente suele ser utilizar el valor **update**. Esto permite que el esquema de la base de datos evolucione a la par de los cambios en el modelo de objetos (como agregar nuevos atributos a `User`) sin perder los datos de prueba ya cargados.

Adicionalmente, se recomienda activar la propiedad **`spring.jpa.show-sql=true`** durante esta etapa para poder visualizar en la consola las sentencias SQL que Hibernate genera automáticamente y así facilitar el proceso de depuración.

3. La interfaz CrudRepository y la jerarquía de repTransaccionesositorios

Inciso 7

¿Qué es CrudRepository? ¿De qué interfaz hereda y qué operaciones provee automáticamente?

Respuesta:

CrudRepository es una **interfaz** fundamental de Spring Data JPA que actúa como el punto de acceso para gestionar la persistencia de datos de una entidad específica. Su propósito principal es eliminar el "**código boilerplate**" (código repetitivo) al permitir que el framework genere automáticamente la implementación del repositorio en tiempo de ejecución a través de proxies dinámicos.

En cuanto a su jerarquía, **CrudRepository** hereda de la interfaz **Repository**, la cual es la interfaz marcadora de más alto nivel dentro del ecosistema de Spring Data.

Esta interfaz provee de forma automática las operaciones estándar de **CRUD** (Create, Read, Update, Delete) sin necesidad de codificación manual. Las operaciones principales que incorpora son:

- **save(S entity)**: Utilizada tanto para crear nuevas instancias como para actualizar las existentes.
- **findById(ID id)**: Permite recuperar una entidad única mediante su clave primaria.
- **findAll()**: Retorna todas las instancias de la entidad almacenadas en el repositorio.
- **deleteById(ID id)**: Elimina una instancia específica de la base de datos.
- **count()**: Devuelve la cantidad total de registros persistidos para esa entidad.
- **existsById(ID id)**: Verifica la existencia de un objeto basándose en su identificador.

Para su utilización en el proyecto, el desarrollador solo debe crear una interfaz que extienda de **CrudRepository**, especificando el **tipo de entidad** y el **tipo de su identificador** (por ejemplo, **CrudRepository<Route, Long>**).

Inciso 8

¿Que agrega cada nivel de la jerarquía respecto del anterior? Describir brevemente las diferencias entre **CrudRepository**, **PagingAndSortingRepository** y **JpaRepository**, indicando que operaciones o capacidades incorpora cada uno

Respuesta:

Dentro del ecosistema de **Spring Data JPA**, la jerarquía de repositorios está diseñada para ofrecer niveles incrementales de funcionalidad, permitiendo que el desarrollador elija la interfaz que mejor se adapte a sus necesidades.

1. CrudRepository

Es el nivel base de la jerarquía funcional y hereda de la interfaz marcadora **Repository**. Su propósito es automatizar las operaciones de persistencia más comunes para una entidad.

- **Capacidades:** Provee operaciones estándar de **CRUD** (Crear, Recuperar, Actualizar y Borrar).
- **Operaciones principales:** Incorpora automáticamente métodos como `save(S entity)`, `findById(ID id)`, `findAll()`, `count()`, `deleteById(ID id)` y `existsById(ID id)`.
- **Uso ideal:** Casos donde solo se requiera una gestión básica de la persistencia sin necesidad de consultas complejas de ordenamiento o paginación.

2. PagingAndSortingRepository

Este nivel hereda de **CrudRepository** y agrega capacidades para el manejo eficiente de grandes volúmenes de datos.

- **Lo que agrega:** Incorpora soporte nativo para **paginación y ordenamiento** mediante el uso de las abstracciones **Pageable** y **Sort**.
- **Operaciones principales:**
 - `findAll(Sort sort)` : Permite recuperar todas las entidades aplicando un criterio de ordenación.
 - `findAll(Pageable pageable)` : Permite recuperar fragmentos de datos (páginas) basándose en el número de página, el tamaño de la misma y el orden deseado.
- **Capacidades:** Permite retornar tipos como **Page<T>** (que incluye el conteo total de elementos) o **Slice<T>** (más liviano, sin conteo total), optimizando la performance en aplicaciones web.

3. JpaRepository

Es la interfaz más completa y específica para tecnologías basadas en JPA (como Hibernate). Hereda de **PagingAndSortingRepository** e incorpora funciones técnicas propias del estándar JPA.

- **Lo que agrega:** Capacidades de control sobre el **Contexto de Persistencia** y operaciones de escritura masiva.
 - **Operaciones principales:**
 - **Sincronización (Flushing):** Métodos como `flush()` para forzar la sincronización de cambios en memoria con la base de datos física.
 - **Borrado masivo:** Provee métodos como `deleteInBatch(Iterable<T> entities)` , que ejecutan una sola sentencia SQL de borrado para múltiples registros, mejorando el rendimiento respecto a los borrados individuales.
 - **Persistencia mejorada:** Métodos como `saveAndFlush(T entity)` para persistir y sincronizar en un solo paso.
 - **Capacidades:** A diferencia de sus predecesores, muchos de sus métodos (como `findAll()`) retornan una `List` en lugar de un `Iterable` , lo que facilita la manipulación de los resultados en Java.
-

En resumen: `CrudRepository` se encarga de lo básico, `PagingAndSortingRepository` añade el control sobre el flujo de datos (páginas/orden), y `JpaRepository` suma herramientas técnicas de bajo nivel para interactuar con el motor de persistencia JPA.

Inciso 9

Crear las interfaces de repositorio para las entidades del modelo extendiendo `CrudRepository`. Indicar los parámetros de tipo correctos (entidad e ID) para cada una:

- `PurchaseRepository`,
- `RouteRepository`,
- `UserRepository`,
- `ServiceRepository`,
- `SupplierRepository` y `ReviewRepository`.

Respuesta:

Revisar código, rama *feat/tp2*

Inciso 10

¿Cómo genera Spring Data JPA la implementación concreta de los repositorios en tiempo de ejecución? ¿Qué rol juega el proxy dinámico de Java en este mecanismo?

Spring Data JPA genera la implementación concreta de los repositorios mediante un proceso de **generación automática en tiempo de ejecución** (runtime), lo que permite que el desarrollador solo deba definir **interfaces** en lugar de clases manuales.

Mecanismo de generación

Cuando la aplicación se inicia, el framework escanea el proyecto en busca de interfaces que extiendan de **Repository** (como **CrudRepository** o **JpaRepository**). Al encontrarlas, Spring Data JPA utiliza la infraestructura de **proxies dinámicos de Java** para crear un objeto real que implemente dicha interfaz en memoria.

El rol del Proxy Dinámico

El **proxy dinámico** juega un papel fundamental como **intermediario e interceptor** de llamadas:

- **Intercepción de mensajes:** Cuando el código de la aplicación (por ejemplo, un servicio) invoca un método de la interfaz del repositorio (como **save()** o un *Query Method* como **findByEmail**), la llamada no va a una clase que tú escribiste, sino que es capturada por el proxy.
- **Delegación de lógica:** El proxy analiza el mensaje recibido. Si es una operación estándar de CRUD, delega el "trabajo sucio" a una implementación base interna que interactúa con la **Session** de Hibernate. Si es un *Query Method*, el proxy traduce el nombre del método a una consulta JPQL/HQL antes de ejecutarla.
- **Transparencia:** Este mecanismo permite que el servicio reciba una instancia funcional del repositorio mediante inyección de dependencias, ocultando completamente el hecho de que no existe una clase física escrita por el programador para ese repositorio.

En resumen, el proxy dinámico permite que Spring Data JPA "**escriba el código por vos**" en tiempo de ejecución, eliminando el código repetitivo (*boilerplate*) que en versiones anteriores se debía codificar manualmente.

Inciso 11

¿Que diferencia hay entre `save()` en Spring Data JPA y `session.save()` / `session.merge()` en Hibernate directo? ¿Cómo decide Spring Data JPA si debe hacer un INSERT o un UPDATE?

Respuesta

La diferencia principal radica en que el método **`save()` de Spring Data JPA** es una **abstracción unificada** diseñada para simplificar el acceso a datos, mientras que en Hibernate directo las operaciones de inserción y actualización están más diferenciadas a nivel de API.

Diferencias entre los métodos

- **`session.save()` (Hibernate):** Se utiliza específicamente para persistir una instancia que se encuentra en estado **transient** (nueva). Este método asigna un identificador al objeto y lo marca para ser insertado en la base de datos durante el próximo *flush*.
- **`session.merge()` (Hibernate):** Su función principal es **actualizar** el estado de una entidad. Se utiliza para tomar un objeto en estado **detached** (desasociado de la sesión) y copiar sus valores sobre una instancia manejada dentro del contexto de persistencia actual.
- **`save()` (Spring Data JPA):** Es un método provisto por la interfaz **`CrudRepository`** que actúa como un **envoltorio inteligente**. En lugar de obligar al desarrollador a elegir entre salvar o fusionar, Spring Data JPA decide internamente qué operación de bajo nivel de Hibernate (o JPA) debe invocar según el estado de la entidad.

¿Cómo decide Spring Data JPA si hacer INSERT o UPDATE?

El framework toma esta decisión basándose en si la entidad se considera "**nueva**" o no. El criterio estándar que utiliza es el siguiente:

1. **Evaluación del Identificador (ID):** Spring Data JPA revisa el atributo anotado con **`@Id`** de la entidad,
 - **INSERT:** Si el identificador es **null** (o 0 en casos de tipos primitivos configurados con **`unsaved-value="0"`**), el framework asume que el

objeto es nuevo y realiza un **INSERT** (llamando internamente a **persist** o **save** de Hibernate).

- **UPDATE:** Si el identificador **ya tiene un valor**, el framework interpreta que la entidad ya existe en el repositorio físico y realiza un **UPDATE** (llamando internamente a **merge**) para sincronizar los cambios realizados en el objeto con la base de datos.

2. **Persistencia por alcance:** Hibernate, al ejecutar el comando final, también aplica el concepto de **persistencia por alcance**, asegurándose de que cualquier objeto nuevo vinculado a la entidad que se está guardando también sea persistido automáticamente si se configuraron las cascadas correspondientes.

En resumen, Spring Data JPA elimina la necesidad de que el programador gestione manualmente los estados *transient* o *detached*, centralizando ambas lógicas en un único mensaje transparente,.

II. Repositorios con Spring Data JPA

1. Migración de repositorios

Inciso 12

Comparar el código de PurchaseRepository de la Práctica 1 (con Session de Hibernate)

con el nuevo PurchaseRepository de Spring Data JPA. ¿Cuántas líneas de código se

eliminaron? ¿Qué operaciones ya no es necesario implementar manualmente?

Respuesta:

Líneas de código eliminadas

- **Versión Vieja (Hibernate):** Aproximadamente **46 líneas** de código (contando declaraciones, métodos, manejo de excepciones y lógica de sesión).
- **Versión Nueva (Spring Data JPA):** Únicamente **6 líneas** de código.
- **Total eliminado:** Se eliminaron cerca de **40 líneas** de código.

Operaciones que ya no es necesario implementar manualmente

Gracias a los mecanismos de **abstracción** que permiten "ocultar detalles de implementación", ya no es necesario realizar las siguientes tareas:

- **Manejo de la Sesión:** Ya no se requiere llamar explícitamente a `this.currentSession()` para interactuar con la base de datos.
- **Creación Manual de Consultas (HQL/JPQL):** En la versión vieja, se debía escribir `createQuery(...)` para cada método. En Spring Data JPA, la consulta se deriva automáticamente del nombre del método (`findAllByUsername`) o se gestiona internamente por el framework.
- **Mapeo de Parámetros:** Se eliminan las llamadas repetitivas a `.setParameter("username", username)`, ya que el framework realiza la **ligadura** de los parámetros reales a los formales de forma automática.
- **Manejo de Excepciones y Logging:** Ya no es necesario envolver cada operación en bloques `try-catch` para capturar `RuntimeException` ni registrar errores manualmente; el sistema de excepciones de Spring traduce y maneja estos eventos de forma nativa.
- **Boilerplate de Implementación:** Se eliminó por completo la necesidad de una clase física (`PurchaseRepositoryImpl`), el constructor y la extensión de `AbstractHibernateRepository`. Ahora solo se define una **interfaz**, lo que representa un nivel de abstracción superior donde se define el "qué" pero se ignora el "cómo".

Inciso 13

En la Práctica 1, los repositorios gestionaban internamente la Session. ¿Que recibe ahora un servicio que necesita usar un repositorio Spring Data? ¿Cómo se declara esa dependencia? ¿Dónde queda ahora la lógica de apertura y cierre de sesiones?

Respuesta

Un servicio que necesita acceder a la base de datos ya no recibe una clase de implementación pesada, sino una **instancia de la interfaz del repositorio** (un objeto "proxy" generado automáticamente por Spring en tiempo de ejecución). Al extender de `CrudRepository`, esta instancia ya posee todos los métodos de persistencia básicos sin que el programador haya tenido que escribir su lógica. La dependencia se declara simplemente como un **atributo de tipo interfaz** dentro de la clase del servicio. Generalmente, se utiliza la **inyección de**

dependencias de Spring para ligar esta interfaz con su implementación real en tiempo de ejecución.

¿Dónde queda ahora la lógica de apertura y cierre de sesiones?

La lógica de gestión de sesiones (que antes se invocaba manualmente con `this.currentSession()`) ahora está **encapsulada y oculta por el framework**.

- **Abstracción de implementación:** Spring Data JPA, junto con el `EntityManager` de JPA, se encarga de abrir y cerrar las sesiones (o contextos de persistencia) de forma transparente para el programador.
- **Gestión por transacciones:** Habitualmente, la "sesión" queda ligada al ciclo de vida de una **transacción**. Esta se suele definir en la capa de servicios (usando la anotación `@Transactional`), de modo que el framework abre la sesión al iniciar el método del servicio y la cierra automáticamente al finalizarlo, asegurando que todos los recursos se liberen correctamente.

2. Query Methods

Inciso 14

¿Cómo funciona la generación de consultas por nombre de método en Spring Data JPA?

Describir las palabras clave principales que puede interpretar (findBy, existsBy, countBy, deleteBy) y cómo se combinan con los atributos de la entidad.

Respuesta

La generación de **Query Methods** es una de las características más potentes de Spring Data JPA, ya que permite que el framework **derive y ejecute consultas automáticamente** interpretando simplemente el nombre de los métodos definidos en la interfaz del repositorio.

Funcionamiento del mecanismo

Cuando la aplicación arranca, Spring Data JPA escanea las interfaces de los repositorios y analiza los nombres de los métodos que siguen ciertas convenciones. El framework actúa de la siguiente manera:

1. **Analiza el prefijo** para determinar el tipo de operación (lectura, conteo, borrado, etc.).
2. **Parsea el resto del nombre** buscando atributos que coincidan con los de la entidad (ej. `email` , `username`).
3. **Genera la consulta JPQL/HQL** correspondiente de forma transparente y asocia los parámetros del método a las condiciones de la consulta en el mismo orden en que aparecen en el nombre.

Palabras clave principales (Prefijos)

Las fuentes identifican cuatro prefijos fundamentales que determinan la intención de la consulta:

- **findBy**: Es el prefijo estándar para las consultas de **recuperación**. Indica al framework que debe generar un `SELECT` para obtener una entidad o una colección de entidades que cumplan los criterios (ej. `findByEmail`).
- **existsBy**: Se utiliza para verificar la **existencia** de registros. En lugar de devolver el objeto, genera una consulta optimizada que retorna un valor booleano (`true` / `false`) indicando si existe al menos una coincidencia.
- **countBy**: Genera una consulta de agregación (`SELECT COUNT`) para obtener la **cantidad total** de entidades que satisfacen las condiciones especificadas.
- **deleteBy**: Se utiliza para realizar operaciones de **borrado derivado**. Elimina físicamente del repositorio los registros que coincidan con los criterios de búsqueda definidos en el nombre del método.

Combinación con atributos y operadores

Para que el método sea funcional, el prefijo debe combinarse con los atributos de la entidad (iniciando con mayúscula) y, opcionalmente, con operadores lógicos y de comparación:

- **Atributos simples**: Se añade el nombre del campo tal cual está en la clase. Ejemplo: `findByUsername(String username)` buscará registros donde el atributo `username` coincida con el parámetro.
- **Operadores lógicos**: Se pueden encadenar múltiples atributos usando **And** u **Or**. Ejemplo: `findByLastNameAndAge` .
- **Comparaciones avanzadas**: Permite usar palabras clave adicionales como **LessThan**, **Between**, **Like** o **OrderBy** para refinar los resultados.

Limitación importante: Esta técnica es ideal para consultas simples sobre uno o dos atributos. Para operaciones que involucren **JOINS complejos**, agrupaciones o subconsultas, es más claro y mantenible utilizar la anotación **@Query** con JPQL explícito.

Inciso 15

¿Cómo se pasan los parametros a un Query Method? ¿Cómo hace Spring Data JPA para asociar cada parámetro del método con la condición correspondiente en la consulta? ¿Qué ocurre si el orden de los parámetros no coincide con el orden de las condiciones en el nombre del método?

Respuesta:

Los parámetros se pasan como **argumentos estándar** en la firma del método definido dentro de la interfaz del repositorio. Por ejemplo, si el método es `findByEmailAndStatus(String email, String status)`, los valores se pasan simplemente invocando al método con los strings correspondientes.

¿Cómo asocia Spring Data JPA cada parámetro con su condición?

El framework utiliza un mecanismo de **asociación por posición**. Al parsear el nombre del método de izquierda a derecha, Spring Data JPA identifica los atributos de la entidad (como `Email` y `Status`) y genera la consulta. Luego, asocia:

- El **primer parámetro** del método a la **primera condición** detectada en el nombre.
- El **segundo parámetro** a la **segunda condición**, y así sucesivamente.

¿Qué ocurre si el orden de los parámetros no coincide?

Si el orden de los argumentos en la firma del método no es idéntico al orden en que aparecen los atributos en el nombre del método, se producirá un **error de vinculación (binding)** o resultados incorrectos:

- **Error de Tipo:** Si los tipos de datos no coinciden (por ejemplo, pasas un `Long` donde el nombre del método sugiere un `String`), la aplicación lanzará una excepción al intentar ejecutar la consulta.
- **Error Lógico:** Si los tipos coinciden pero los valores están invertidos (por ejemplo, en `findByFirstNameAndLastName(String lastName, String firstName)`), la base de datos buscará el nombre en la columna de

apellido y viceversa, lo que resultará en que la consulta **no devuelva registros** o devuelva datos erróneos.

Nota: A diferencia de las consultas con la anotación `@Query`, donde se pueden usar parámetros nombrados (`:name`), en los **Query Methods derivados** (por nombre) la relación es estrictamente **posicional**.

Inciso 16

Investigar y para cada uno de los siguientes requerimientos indicar si es posible resolverlo con un Query Method y escribir la firma del método correspondiente. Si no es posible, explicar por qué:

1. Buscar todas las Purchase de un usuario dado su username.
2. Verificar si existe alguna Purchase para una Route dada.
3. Contar cuantas Review tienen un rating mayor o igual a un valor dado.
4. Obtener todas las Route cuyo precio sea menor a un valor dado, ordenadas por nombre.
5. Buscar un User por su email.
6. Obtener los top 3 Route con mayor cantidad de Purchase.

Inciso 16.1

Buscar todas las Purchase de un usuario dado su username

- **¿Es posible?** Sí. Spring Data JPA permite navegar por las relaciones de una entidad (en este caso, de `Purchase` hacia `User`) usando el nombre del atributo seguido de la propiedad deseada.
- **Repositorio:** `PurchaseRepository`
- **Firma:** `List<Purchase> findByUserUsername(String username);`

Inciso 16.2

Verificar si existe alguna Purchase para una Route dada

- **¿Es posible?** Sí. Se utiliza el prefijo `existsBy` diseñado específicamente para retornar un valor booleano de forma optimizada.
- **Repositorio:** `PurchaseRepository`

- **Firma:** `boolean existsByRoute(Route route);` (o `existsByRouteId(Long routeId)`)

Inciso 16.3

Contar cuántas Review tienen un rating mayor o igual a un valor dado

- **¿Es posible?** Sí. Se combina el prefijo **countBy** con la palabra clave **GreaterThanEqual** que el framework sabe interpretar para generar la condición `>=`.
- **Repositorio:** `ReviewRepository`
- **Firma:** `long countByRatingGreaterThanEqual(int rating);`

Inciso 16.4

Obtener todas las Route cuyo precio sea menor a un valor dado, ordenadas por nombre

- **¿Es posible?** Sí. Es posible encadenar condiciones de comparación como **LessThan** con instrucciones de ordenamiento como **OrderBy...Asc/Desc** directamente en el nombre.
- **Repositorio:** `RouteRepository`
- **Firma:** `List<Route> findByPriceLessThanOrderByNameAsc(float price);`

Inciso 16.5

Buscar un User por su email

- **¿Es posible?** Sí. Es una consulta de búsqueda directa sobre un atributo simple de la entidad `User`.
- **Repositorio:** `UserRepository`
- **Firma:** `Optional<User> findByEmail(String email);` (o simplemente `User` si se asume que el email es único).

Inciso 16.6

Obtener los top 3 Route con mayor cantidad de Purchase

- **¿Es posible?** No.
- **Explicación:** Aunque Spring Data JPA soporta la palabra clave `Top`, este requerimiento exige una **agregación** (contar cuántas compras tiene cada

ruta) combinada con un **agrupamiento** (`GROUP BY`) y un ordenamiento basado en el resultado de esa cuenta. Los Query Methods están limitados a consultas sobre atributos existentes y no pueden derivar lógicas que involucren funciones de agregación complejas o agrupamientos para el ordenamiento.

- **Solución recomendada:** Para este caso es necesario utilizar la anotación `@Query` con una consulta JPQL explícita que realice el conteo y el ordenamiento de forma manual.

Inciso 17

Investigar cuales son las palabras clave (keywords) disponibles en Spring Data JPA para construir Query Methods. ¿Qué tipos de condiciones, comparaciones y operadores lógicos soporta?

Respuesta

Prefijos de Intención

Determinan la operación base que se realizará sobre la base de datos:

- **findBy / readBy / getBy / queryBy:** Inician una consulta de recuperación de datos (SELECT).
- **existsBy:** Genera una consulta optimizada que retorna un booleano para verificar la existencia de registros.
- **countBy:** Realiza una consulta de agregación para contar la cantidad de registros (SELECT COUNT).
- **deleteBy / removeBy:** Ejecuta una operación de borrado derivado sobre los registros que cumplan la condición.

Operadores Lógicos

Permiten combinar múltiples condiciones en una misma consulta:

- **And:** Une dos condiciones que deben cumplirse simultáneamente (ej. `findByEmailAndStatus`).
- **Or:** Une condiciones donde basta que se cumpla una de ellas (ej. `findByLastNameOrFirstName`).

Palabras Clave de Comparación

Spring Data JPA soporta diversos operadores para refinar las búsquedas sobre los atributos:

- **Igualdad (por defecto):** Si no se agrega una keyword, se asume la igualdad (ej. `findByUsername`).
- **Comparaciones Numéricas y de Fechas:**
 - **LessThan / LessThanEqual:** Menor que / Menor o igual que.
 - **GreaterThan / GreaterThanEqual:** Mayor que / Mayor o igual que.
 - **Between:** Valores dentro de un rango inclusivo.
- **Comparación de Cadenas (Strings):**
 - **Like / NotLike:** Búsqueda por patrón (usualmente requiere comodines manuales).
 - **StartingWith / EndingWith / Containing:** Coincidencias al inicio, al final o en cualquier parte de la cadena, respectivamente.
- **Colecciones y Nulos:**
 - **IsNull / IsNotNull:** Verifica si un campo es nulo o no.
 - **In / NotIn:** Verifica si el valor se encuentra dentro de una lista de parámetros.

Ordenamiento y Limitación

- **OrderBy:** Permite definir el orden de los resultados seguido del nombre del atributo y la dirección (`Asc` o `Desc`). Ejemplo:
`findByPriceLessThanOrderByNameAsc` .
- **First / Top:** Se utilizan después del prefijo para limitar la cantidad de resultados devueltos (ej. `findFirst3ByOrderByPriceAsc` para obtener los 3 más baratos).

Inciso 18

¿Qué limitaciones tienen los Query Methods? Describir al menos tres casos del modelo donde esta técnica resulta insuficiente y es necesario otro enfoque. Describa los otros enfoques posibles para implementar dichas consultas.

Respuesta

Aunque son muy potentes para filtros sobre atributos existentes, resultan insuficientes cuando la consulta requiere **agregaciones complejas** (como

GROUP BY), **JOINS manuales** o lógicas de negocio muy específicas. En esos casos, las fuentes recomiendan el uso de la anotación **@Query** con JPQL explícito.

3. Consultas con @Query

Inciso 19

En la Práctica 1 se utilizaron consultas HQL (Hibernate Query Language).
¿Que diferencia hay entre HQL y JPQL (Java Persistence Query Language)?
¿Son intercambiables? ¿Cual de los dos acepta @Query por defecto?

Respuesta:

Diferencias entre HQL y JPQL

- **Definición:** JPQL (Java Persistence Query Language) es el lenguaje de consultas definido por la especificación **JPA**, que es el contrato estándar de persistencia en Java. **HQL** (Hibernate Query Language), por su parte, es el lenguaje propio de **Hibernate**, la implementación concreta que utilizamos.
- **Alcance:** HQL es un **superset de JPQL**. Esto significa que HQL incluye todas las capacidades de JPQL pero añade funcionalidades adicionales y mayor flexibilidad que no están contempladas en el estándar estricto de Java.
- **Abstracción:** Ambos operan sobre el **modelo de objetos** (clases, atributos y relaciones) en lugar de tablas y columnas físicas, y Hibernate se encarga de traducirlos al dialecto SQL específico del motor de base de datos.

¿Son intercambiables?

En gran medida, **sí son intercambiables** para consultas estándar, ya que Hibernate (el motor que corre por debajo) es capaz de interpretar ambos lenguajes. Sin embargo:

- Si escribes una consulta siguiendo estrictamente el estándar **JPQL**, tu código será más portable hacia otras implementaciones de JPA.
- Si utilizas funciones avanzadas exclusivas de **HQL**, la consulta podría no funcionar si en el futuro se cambia Hibernate por otro proveedor de persistencia.

¿Cuál acepta **@Query** por defecto?

En el contexto de **Spring Data JPA**, la anotación **@Query** acepta JPQL por defecto.

Aunque Spring Data JPA utiliza Hibernate internamente y este puede procesar HQL, el framework está diseñado para alinearse con el estándar de la industria (JPA), por lo que se asume que las cadenas de consulta proporcionadas en la anotación son JPQL a menos que se especifique que es una consulta nativa mediante el parámetro `nativeQuery = true`.

Inciso 20

Que diferencia hay entre una consulta @Query con JPQL y una con `nativeQuery = true`?

¿Cuándo conviene cada una? Dar un ejemplo concreto del modelo para cada caso.

Respuesta:

@Query con JPQL (Java Persistence Query Language)

Es el lenguaje de consultas estándar de JPA y es la opción por defecto en Spring Data JPA.

- **Diferencia clave:** Se escribe utilizando los **nombres de las clases (entidades) y sus atributos**, no los nombres de las tablas y columnas físicas.
- **Cuándo conviene:** Siempre que sea posible, ya que mantiene la **independencia del motor de base de datos** (portabilidad) y permite que el ORM gestione automáticamente la transformación de tipos y relaciones. Es ideal para navegar entre asociaciones de objetos sin preocuparse por los JOINS físicos de bajo nivel.
- **Ejemplo concreto del modelo:** Buscar todas las compras de un usuario por su nombre de usuario.
 - `@Query("SELECT p FROM Purchase p WHERE p.user.username = :username")`.
 - *Nota:* Aquí se usa `Purchase` (clase) y `user.username` (navegación de objetos), no los nombres de tabla `ITEM_PURCHASE` o columnas `FK_USER`.

@Query con `nativeQuery = true`

Esta opción permite enviar sentencias en el lenguaje **SQL nativo** del motor de base de datos configurado (ej. MySQL, Oracle, HSQL).

- **Diferencia clave:** Se debe escribir utilizando los **nombres reales de las tablas y columnas** de la base de datos física. No tiene conocimiento del modelo de objetos.
- **Cuándo conviene:**
 - Para realizar consultas que requieren **funciones específicas** del motor de base de datos que JPQL no soporta.
 - Para optimizar el rendimiento en consultas extremadamente complejas donde el SQL generado por el ORM no es eficiente.
 - Para **eludir filtros globales** (como la anotación `@Where` utilizada en el **borrado lógico**), permitiendo recuperar registros que el ORM normalmente ignoraría.
- **Ejemplo concreto del modelo:** Consultar todos los usuarios de la tabla, incluyendo aquellos marcados como borrados lógicamente (ignorando el filtro `@Where("is_deleted = false")`).
 - `@Query(value = "SELECT * FROM ITEM_USER", nativeQuery = true)` .
 - *Nota:* Aquí se usa el nombre físico de la tabla `ITEM_USER` y se recuperan todas las filas físicamente presentes en el disco.

Inciso 21

¿Cómo se pasan parámetros a una consulta `@Query`? Describir y comparar las dos formas: parámetros posicionales (`?1`, `?2`) y parámetros nombrados (`@Param`). ¿Cuál es la forma recomendada y por qué?

Respuesta

En la utilización de la anotación `@Query` en Spring Data JPA, existen dos formas principales de pasar parámetros desde el método del repositorio hacia la consulta JPQL/HQL.

Parámetros Posicionales (`?1` , `?2`)

En esta modalidad, los parámetros se asocian según su **orden de aparición** en la firma del método.

- **Sintaxis:** Dentro de la cadena de consulta se utiliza el prefijo `?` seguido de un número que indica la posición del argumento (empezando por 1).
- **Ejemplo:** `SELECT u FROM User u WHERE u.email = ?1 AND u.status = ?2 .`
- **Mecanismo:** Spring Data JPA toma el primer argumento del método y lo coloca en el lugar de `?1` , el segundo en `?2` , y así sucesivamente.

Parámetros Nombrados (`@Param`)

Esta forma permite una vinculación explícita mediante nombres descriptivos, independientemente del orden de los argumentos.

- **Sintaxis:** En la consulta se utiliza un nombre precedido por dos puntos (ej. `:email`). En el método del repositorio, se debe anteponer la anotación `@Param("email")` al argumento correspondiente para que el framework pueda realizar la asociación.
- **Ejemplo:** `@Query("SELECT u FROM User u WHERE u.email = :email")
User findByEmail(@Param("email") String email);`

La forma recomendada es el uso de **parámetros nombrados** (`**@Param**`).

Razones de la recomendación:

- **Mantenibilidad:** Evitan errores accidentales cuando se refactoriza el código. Si un desarrollador cambia el orden de los parámetros en el método o agrega uno nuevo en el medio, la consulta con parámetros nombrados seguirá funcionando correctamente, mientras que la posicional fallará o producirá resultados erróneos.
- **Claridad del código:** Al usar nombres como `:username` o `:startDate` , la consulta es autodocumentada y mucho más fácil de entender y depurar que una llena de índices como `?1` , `?2` , `?3` .
- **Eficiencia en consultas complejas:** Permiten reutilizar el mismo valor en diferentes partes de una consulta larga (por ejemplo, en subconsultas o múltiples condiciones) sin necesidad de pasar el mismo valor varias veces como argumento del método.

4. Paginación y ordenamiento

Inciso 22

¿Qué es la interfaz Pageable? ¿Cómo se construye una instancia de Pageable?
¿Qué información encapsula (número de página, tamaño, ordenamiento)?

Respuesta

La interfaz **Pageable** está diseñada para gestionar de forma nativa la **paginación y el ordenamiento** de los resultados de una consulta, eliminando la necesidad de modificar manualmente las sentencias JPQL o HQL.

¿Qué es la interfaz Pageable?

Es una abstracción que permite solicitar fragmentos específicos de datos (páginas) de un repositorio. Cuando se añade como parámetro a un método de búsqueda (ej. `findAll(Pageable pageable)`), Spring Data JPA intercepta la llamada y aplica automáticamente las cláusulas de limitación y desplazamiento (`limit` y `offset`) correspondientes en el motor de base de datos.

¿Cómo se construye una instancia?

Aunque Pageable es una interfaz, para utilizarla se debe instanciar una clase que la implemente. La forma estándar y recomendada es mediante el método de fábrica estático `PageRequest.of(...)`.

Información que encapsula

Un objeto Pageable agrupa de forma compacta tres criterios fundamentales para la recuperación de datos:

- Número de página (Page Number): Es el índice de la página que se desea recuperar. Es importante notar que, en Spring Data, este índice es base cero (la primera página es la 0).
- Tamaño de la página (Size): Define la cantidad máxima de registros o instancias que deben incluirse en cada página (por ejemplo, 10, 20 o 50 resultados por vez).
- Ordenamiento (Sort): Es un parámetro opcional que permite definir bajo qué atributos y en qué dirección (Ascendente o Descendente) se deben organizar los registros antes de ser paginados.

Inciso 23

¿Que diferencia hay entre los tipos de retorno Page< T > y Slice< T >? ¿Cuándo conviene cada uno? ¿Qué consulta adicional ejecuta Page< T > que Slice< T > no ejecuta?

Respuesta:

- **Page< T >**: Es un objeto completo que contiene los datos de la página actual, pero además conoce el **total de elementos** disponibles en la base de datos y la **cantidad total de páginas**.
- **Slice< T >**: Es una versión más liviana que solo contiene los datos de la página actual y sabe **si existe una página siguiente**, pero desconoce el conteo total de registros o páginas.

Conveniencia de cada uno:

- **Conviene usar Page< T >** cuando la interfaz de usuario requiere una **barra de paginación clásica** (con números de página, botón de "Última página" o indicador de "Total de resultados").
- **Conviene usar Slice< T >** cuando se implementan mecanismos de "**Scroll Infinito**" o botones de "**Cargar más**", donde al usuario no le interesa saber cuántas páginas faltan, sino simplemente seguir navegando.

Consulta adicional

Para poder informar el total de elementos y páginas, **Page< T >** ejecuta automáticamente una consulta adicional de tipo **SELECT COUNT** sobre la base de datos. Por el contrario, **Slice< T >** no ejecuta esta consulta de conteo; simplemente intenta recuperar un registro más de los solicitados (por ejemplo, pide 11 registros si el tamaño es 10) para determinar si hay contenido pendiente para una próxima entrega.

Inciso 24

Mostrar como se invocará desde la capa de servicio un método paginado para obtener la segunda página de compras de un usuario, con 10 resultados ordenados por fecha descendente.

Respuesta

Para invocar un método paginado desde la capa de servicio cumpliendo con los requisitos de obtener la **segunda página**, con **10 resultados** y ordenados

por **fecha descendente**, se debe utilizar la interfaz `Pageable` y la clase de fábrica `PageRequest`.

A continuación, se muestra cómo se realizaría la implementación en la capa del repositorio (ej. `PurchaseRepository`):

```
public interface PurchaseRepository extends
JpaRepository<Purchase, Long> {
    Page<Purchase> findByUserOrderByDateDesc(User user,
Pageable pageable);
}
```

En el servicio, simplemente construimos el objeto `Pageable` con el índice de página y el tamaño. Como el orden ya lo resuelve el repositorio por su nombre, no es necesario pasarle un objeto `Sort` adicional al `PageRequest`.

```
@Service
public class PurchaseServiceImpl implements PurchaseService {

    @Autowired
    private PurchaseRepository purchaseRepository;

    public Page<Purchase> getRecentPurchasesByUser(User user)
{
    // Página 1 (es la segunda página, ya que el índice
inicia en 0)
    // Tamaño 10 resultados
    Pageable pageable = PageRequest.of(1, 10);

    // Invocación al método que ya devuelve los datos
ordenados DESC
    return
purchaseRepository.findByUserOrderByDateDesc(user, pageable);
}
}
```

Inciso 25

¿Cómo se agrega ordenamiento a un Query Method sin usar Pageable? ¿Qué palabra clave se usa en el nombre del método? Mostrar un ejemplo concreto con el modelo

Respuesta:

Para agregar ordenamiento a un **Query Method** sin utilizar la interfaz **Pageable**, se debe incluir la lógica de orden directamente en el nombre del método definido en la interfaz del repositorio.

La palabra clave que se utiliza es **OrderBy**, seguida del nombre del atributo de la entidad por el cual se desea ordenar y, opcionalmente, la dirección del ordenamiento (**Asc** para ascendente o **Desc** para descendente).

Ejemplo concreto con el modelo

Si quisiéramos obtener todas las rutas (**Route**) cuyo precio sea menor a un valor dado, ordenadas alfabéticamente por su nombre, la firma del método en el repositorio sería la siguiente:

```
public interface RouteRepository extends CrudRepository<Route, Long> {  
    // Ejemplo de ordenamiento fijo por nombre ascendente  
    List<Route> findByPriceLessThanOrderByNameAsc(float price);  
}
```

5. Implementación de las consultas del modelo

Inciso 26

Para cada consulta, indicar la estrategia elegida (Query Method o @Query) e implementarla en el repositorio correspondiente:

Consulta	Repositorio	Decisión
getAllPurchasesOfUsername(String username)	PurchaseRepository	@Query
getUsersSpendingMoreThan(float amount)	UserRepository	@Query
getTopNSuppliersInPurchases(int n)	SupplierRepository	@Query
getCountOfPurchasesBetweenDates(Date from, Date to)	PurchaseRepository	@Query

Consulta	Repositorio	Decisión
<code>getRoutesWithStop(Stop stop)</code>	RouteRepository	<code>@Query</code>
<code>getMaxStopOfRoutes()</code>	RouteRepository	<code>@Query</code>
<code>getRoutesNotSell()</code>	RouteRepository	<code>@Query</code>
<code>getTop3RoutesWithMaxRating()</code>	RouteRepository	<code>@Query</code>
<code>getMostDemandedService()</code>	ServiceRepository	<code>@Query</code>
<code>getTourGuidesWithRating1()</code>	UserRepository	<code>@Query</code>

Consulta	Justificación
<code>getAllPurchasesOfUsername(String u)</code>	Carece de la palabra clave <code>By</code> (podemos cambiarle el nombre, de la entidad mapear "AllPurchases" a "AllPurchasesByUser" y usar <code>@Query</code>).
<code>getUsersSpendingMoreThan(float a)</code>	Además de no tener el <code>By</code> , requiere una consulta compleja (<code>SUM(gastos) > amount</code>), por lo que este tipo de agregaciones numéricas no se soportan.
<code>getTopNSuppliersInPurchases(int n)</code>	Los Query Methods soportan un límite máximo de número en el nombre del método. No soportan parametrizar ese límite máximo con una variable entera <code>int n</code> por parámetro (aunque se puede usar la interfaz <code>Pageable</code>).
<code>getCountOfPurchasesBetweenDates(...)</code>	No tiene el formato estándar (falta la palabra clave <code>Query Method</code> debería llamarse <code>getCountOfPurchasesBetweenDatesFrom to</code>).
<code>getRoutesWithStop(Stop stop)</code>	Carece del formato estándar. El nombre de la propiedad literal llamada "Route" no indica la relación. Si fuera un Query Method debería llamarse <code>findByStopsContaining</code> .
<code>getMaxStopOfRoutes()</code>	Carece de <code>By</code> . Además, obtener el máximo de una lista anidada usando un Query Method requiere una agregación explícita en un <code>@Query</code> .
<code>getRoutesNotSell()</code>	No contiene el <code>By</code> . Semánticamente no tiene relación con los relacionados que sean nulos o no. Debería llamarse <code>findByPurchasesIsNull</code> .
<code>getTop3RoutesWithMaxRating()</code>	Aunque prefijos como <code>Top3</code> son comunes, tienen el mismo problema: la falta de la palabra clave <code>Query Method</code> .

Consulta	Justificación
	criterios, sumado a la complejidad por rating (<code>MAX(rating)</code>) o similar.
<code>getMostDemandedService()</code>	Obtener el "más demandado" requiere un ordenamiento por conteo (<code>ORDER BY COUNT(*)</code>) sobre el registro resultante. Operaciones que requieren <code>@Query</code> .
<code>getTourGuidesWithRating1()</code>	No contiene <code>By</code> . Además, al tener un literal al final (<code>1</code>), Spring tratará "Rating1" a menos que todo se provea el JPQL asociado explícitamente.

Inciso 27

Implementar cada una de las consultas de la tabla anterior en el repositorio correspondiente, utilizando la estrategia indicada. Para las consultas que requieran paginación, utilizar Pageable como parámetro adicional.

Ver el código

Inciso 28

Comparar la implementación de al menos dos consultas complejas (por ejemplo `getTopNSuppliersInPurchases` y `getTop3RoutesWithMaxRating`) entre la Práctica 1 y esta práctica. ¿Qué diferencias hay en términos de cantidad de código, legibilidad y acoplamiento a la infraestructura de persistencia? A partir de esta comparación, ¿qué es el código boilerplate y qué ventaja concreta representa su reducción para el mantenimiento de un proyecto real?

Comparación de Implementaciones

Cantidad de Código

- **Práctica 1 (Hibernate directo):** Requiere una **clase concreta** para el repositorio donde se debe inyectar la `SessionFactory` . Cada método necesita bloques `try-catch` manuales para el manejo de excepciones y logueo, además de llamadas explícitas a la API de Hibernate como `.createQuery()` , `.setMaxResults()` y `.getResultList()` .

- **Práctica 2 (Spring Data JPA):** Solo requiere una **interfaz**. Al extender de `JpaRepository` o `CrudRepository`, no es necesario escribir código para las operaciones básicas ni para la gestión de la sesión. La consulta compleja se reduce simplemente a la anotación `@Query` sobre la firma del método.

Legibilidad

- **Práctica 1:** La lógica de negocio (la consulta HQL) queda "sepultada" bajo código de infraestructura. El desarrollador debe leer líneas de manejo de errores y llamadas a métodos de la `Session` antes de llegar a la intención real del método.
- **Práctica 2:** Es mucho más **declarativa**. La intención es clara: se define una consulta y se asocia a un método. El uso de bloques de texto (`"""`) permite que el JPQL sea el protagonista, mejorando la comprensión inmediata de qué datos se están recuperando.

Acoplamiento a la infraestructura

- **Práctica 1:** Existe un **alto acoplamiento**. El repositorio es una clase que depende directamente de clases específicas de Hibernate (`Session`, `Query`, `HibernateException`). Si se quisiera cambiar el motor de persistencia, habría que reescribir toda la clase.
- **Práctica 2:** El acoplamiento es **mínimo**. El repositorio es una interfaz, y la implementación real es generada en tiempo de ejecución mediante un **proxy dinámico** de Java. La capa de servicio solo conoce la interfaz, lo que permite que Spring Data JPA actúe como una capa de abstracción sobre el ORM subyacente.

Código Boilerplate y su reducción

¿Qué es el código boilerplate? Es el código repetitivo que debe incluirse en muchos lugares con poca o ninguna variación para que la infraestructura funcione, pero que **no aporta valor a la lógica de negocio**. Ejemplos claros en la Práctica 1 son la apertura/cierre de sesiones, el manejo de transacciones manuales y la repetición de estructuras `try-catch` para operaciones de base de datos.

Ventajas concretas de su reducción:

1. **Mantenibilidad:** Menos líneas de código significan menos lugares donde pueden aparecer errores.
2. **Productividad:** El desarrollador se enfoca exclusivamente en el "qué" (la lógica del dominio) y no en el "cómo" (los detalles técnicos de la persistencia).
3. **Evolución del proyecto:** Al delegar la infraestructura al framework, es más fácil actualizar versiones o realizar cambios globales de configuración (vía `application.properties`) sin tocar cada repositorio individualmente.

III. Transacciones

1. Aspectos avanzados de @Transactional

Inciso 29

¿Cuál es el atributo `readOnly = true` en `@Transactional`? ¿Qué optimizaciones activa?

¿En qué métodos del servicio conviene aplicarlo?

Respuesta:

El atributo **`readOnly = true`** en la anotación `@Transactional` de Spring es una indicación (hint) que se le da al framework y al motor de persistencia subyacente (como Hibernate) de que la unidad de trabajo no realizará modificaciones en los datos.

¿Qué optimizaciones activa?

- **Eliminación del Dirty Checking (Cotejo de suciedad):** Hibernate, al saber que la transacción es de solo lectura, **no necesita mantener copias de seguridad (snapshots)** del estado inicial de las entidades ni comparar sus atributos al finalizar la transacción para detectar cambios. Esto reduce significativamente el consumo de **memoria y CPU**.
- **Gestión del Flush Mode^{**}:** El modo de descarga de la sesión se configura automáticamente como `MANUAL` o `NEVER`. Esto evita que el framework intente sincronizar innecesariamente el estado de los objetos en memoria con la base de datos al cerrar la sesión, ahorrando operaciones de red.
- **Optimizaciones a nivel de base de datos:** En muchos motores de base de datos, marcar una transacción como `readOnly` permite que el optimizador de consultas elija planes de ejecución más eficientes y

gestione los **bloqueos (locks)** de forma menos restrictiva, mejorando la concurrencia.

¿En qué métodos del servicio conviene aplicarlo?

Siguiendo los principios de **transparencia y eficiencia** discutidos en las fuentes, conviene aplicar `readOnly = true` en:

- **Métodos de búsqueda y recuperación:** Todos aquellos métodos que solo ejecutan consultas `SELECT` , como por ejemplo:
 - `findAll()`
 - `findById()`
 - Métodos que devuelven listas paginadas o filtradas (ej. `getAllPurchasesOfUsername()`).
- **Consultas complejas de agregación:** Métodos que realizan cálculos estadísticos o conteos (ej. `getCountOfPurchasesBetweenDates` , `getTop3RoutesWithMaxRating`).

Conclusión

Para mantener la arquitectura limpia , se recomienda seguir este patrón en la **capa de servicio**:

1. Anotar la **clase** de servicio completa con `@Transactional(readOnly = true)` . De esta manera, por defecto, todos los métodos serán de solo lectura.
2. Anotar específicamente con `@Transactional` (sin el atributo o con `readOnly = false`) solo los métodos que realicen **escritura, actualización o borrado** (ej. guardar un servicio, actualizar un precio o eliminar una ruta).

Inciso 30

¿Cómo maneja Spring el rollback automático? ¿Sobre qué tipos de excepción hace rollback por defecto? ¿Cómo se configura para el rollback? Dar un ejemplo con el modelo.

En el marco de **Spring Data JPA** y la gestión declarativa de transacciones mediante la anotación `@Transactional`, el manejo del rollback automático funciona de la siguiente manera:

1. ¿Cómo maneja Spring el rollback automático?

Spring utiliza el patrón **Proxy**. Cuando un método de la capa de servicio anotado con `@Transactional` es invocado, Spring crea un interceptor que abre la transacción.

- Si el método **termina exitosamente**, el interceptor solicita el `commit` a la base de datos.
- Si el método **lanza una excepción** que cumple con los criterios de rollback, el interceptor captura la excepción y solicita automáticamente la operación `rollback()` al manejador de transacciones para deshacer todos los cambios realizados en esa unidad de trabajo.

2. ¿Sobre qué tipos de excepción hace rollback por defecto?

Es fundamental distinguir el comportamiento predeterminado de Spring:

- **Excepciones no comprobadas (Unchecked Exceptions):** Spring realiza el rollback automático **por defecto** solo cuando se lanzan subclases de `RuntimeException` (como `NullPointerException`, `IllegalArgumentException` o las excepciones de persistencia de Spring) y errores de tipo `Error`.
- **Excepciones comprobadas (Checked Exceptions):** Por defecto, Spring **no realiza rollback** ante excepciones que heredan directamente de `Exception` (pero no de `RuntimeException`). Estas se consideran excepciones de "negocio" que el desarrollador debe manejar sin invalidar necesariamente la transacción.

3. ¿Cómo se configura para el rollback?

Se puede personalizar este comportamiento utilizando los atributos de la anotación `@Transactional`:

- **rollbackFor:** Permite especificar una o varias clases de excepciones (incluso *Checked Exceptions*) ante las cuales se debe forzar el rollback. Ejemplo: `@Transactional(rollbackFor = Exception.class)`.
- **noRollbackFor:** Indica excepciones específicas ante las cuales **no** se debe realizar el rollback, incluso si son de tipo `RuntimeException`.

4. Ejemplo con el modelo

Imaginemos un método en `ToursServiceImpl` para registrar una compra (`Purchase`). Queremos que si ocurre cualquier error de negocio (como falta de

stock) o un error inesperado, no se guarde nada.

```
@Service
public class ToursServiceImpl implements ToursService {

    @Autowired
    private PurchaseRepository purchaseRepository;

    @Override
    @Transactional(rollbackFor =
{RouteCapacityException.class}) // Forzamos rollback para esta
checked exception
    public void confirmPurchase(Purchase purchase) throws
RouteCapacityException {
        // 1. Se guarda la compra (operación de escritura)
        purchaseRepository.save(purchase);

        // 2. Lógica de negocio: Verificar capacidad de la
ruta
        if (purchase.getRoute().isFull()) {
            // Al lanzar esta excepción, Spring detecta el
error y hace ROLLBACK automático
            // de la compra guardada en el paso 1.
            throw new RouteCapacityException("La ruta ya no
tiene cupo disponible");
        }
    }
}
```

En este ejemplo, si se lanza `RouteCapacityException`, Spring asegurará que la `Purchase` no quede persistida en la base de datos, manteniendo la **atomicidad** de la operación.

Inciso 31

Revisar y actualizar la capa de servicio de la Práctica 1 para que utilice los repositorios Spring Data JPA. Reemplazar todas las llamadas directas a la Session (session.save, session.get, session.createQuery, etc.) por las operaciones equivalentes de los repositorios. La lógica de negocio y las anotaciones `@Transactional` ya existentes no deben modificarse salvo que sea necesario por los cambios anteriores.

IV. DTOs y Proyecciones

1. ¿Qué es un DTO y cuando usarlo?

Inciso 32

¿Que es un DTO (Data Transfer Object)? ¿Cuál es su propósito dentro de la arquitectura de una aplicación? ¿Por qué no siempre conviene devolver una entidad JPA directamente desde la capa de acceso a datos?

Respuesta:

Un **DTO (Data Transfer Object)** es un objeto diseñado exclusivamente con el propósito de **transferir datos o información** entre diferentes capas de una aplicación. A diferencia de los objetos del modelo (entidades), los DTO no contienen lógica de negocio y están compuestos únicamente por **atributos con sus respectivos métodos getter y setter**.

Propósito dentro de la arquitectura

El uso de DTOs cumple varios objetivos estratégicos en la arquitectura de un sistema:

- **Intercambio de información entre capas:** Actúan como el medio de comunicación estándar entre la capa Web (interfaz) y la capa de Servicios, permitiendo que la información fluya sin exponer la complejidad interna del modelo.
- **Separación de responsabilidades:** Refuerzan la división de capas al impedir que capas superiores invoquen comportamiento o lógica de negocio directamente sobre los objetos que reciben, ya que los DTOs carecen de dicho comportamiento.
- **Lectura fuera del contexto transaccional:** Permiten que la información relevante de las entidades persistentes sea cargada en objetos volátiles antes de cerrar una transacción. Esto garantiza que la información sea accesible incluso después de que la unidad de trabajo (transacción) haya finalizado.
- **Optimización del tráfico de datos:** Permiten que la aplicación envíe únicamente un **subconjunto de datos** necesario para un caso de uso específico, en lugar de transferir objetos completos con todas sus relaciones.

¿Por qué no conviene devolver entidades JPA directamente?

Exponer las entidades del modelo directamente desde la capa de acceso a datos conlleva diversos riesgos técnicos y de diseño:

1. **Límites de la Transacción y Lazy Loading:** Las entidades JPA a menudo tienen relaciones configuradas como "perezosas" (*lazy*). Si se intenta acceder a estas relaciones fuera del marco transaccional (por ejemplo, en la capa de vista), el sistema fallará lanzando una excepción porque no está permitido leer información de la base de datos sin una transacción activa.
2. **Ciclos de Serialización:** Al convertir entidades en formatos como JSON para endpoints web, las relaciones bidireccionales pueden generar bucles infinitos (ciclos de serialización) si el framework intenta recorrer las asociaciones de ida y vuelta constantemente.
3. **Acoplamiento entre Capas:** Devolver entidades acopla la interfaz de usuario directamente al esquema de la base de datos. Cualquier cambio menor en la estructura de una tabla obligaría a modificar la lógica de presentación.
4. **Seguridad y Exposición de Lógica:** Las entidades contienen lógica de negocio y, en ocasiones, datos sensibles o innecesarios para el cliente. Al devolver la entidad completa, el desarrollador corre el riesgo de que el cliente intente ejecutar métodos del modelo en forma indebida o acceda a información privada.
5. **Conflictos de Concurrencia:** Recuperar grafos de objetos demasiado grandes a memoria aumenta la probabilidad de que surjan conflictos de versiones si otros usuarios modifican esos mismos datos simultáneamente. Los DTOs, al ser copias desconectadas, eliminan este riesgo de manipulación indebida en memoria.

Inciso 33

Describir dos situaciones concretas del modelo donde devolver un DTO sería más

adecuado que devolver la entidad completa. Para cada caso indicar qué campos

contendría el DTO y por qué.

Respuesta

Basado en el modelo de gestión de tours y la arquitectura de capas, a continuación se describen dos situaciones donde el uso de un **DTO** es más adecuado que devolver la entidad completa:

*Listado de Catálogo de Rutas (**RouteSummaryDTO**)*

Esta situación ocurre cuando la aplicación necesita mostrar una lista de todos los recorridos disponibles para que un cliente elija uno, sin necesidad de ver detalles técnicos o históricos profundos en ese momento.

- **Campos del DTO:**

- **id** y **name** : Para identificar y mostrar el nombre de la ruta.
- **price** : Para que el usuario conozca el costo del tour.
- **totalKm** : Para dar una idea de la extensión del recorrido.
- **stopCount** : Un campo calculado (entero) que indique la cantidad de paradas, en lugar de la colección completa de objetos **Stop** .

- **¿Por qué es más adecuado?** La entidad **Route** tiene relaciones complejas con **Stop** , **DriverUser** , **TourGuideUser** y una lista potencialmente larga de **Purchase** . Si se devuelve la entidad completa, el sistema intentaría cargar (o fallaría al hacerlo fuera de la transacción) todas estas asociaciones, generando un tráfico de datos innecesario y posibles excepciones de **LazyInitializationException** al intentar acceder a las paradas o guías en la capa de vista. Un DTO permite enviar solo la información mínima para la toma de decisiones del usuario, optimizando el rendimiento.

*Autenticación y Perfil de Usuario (**UserProfileDTO**)*

Esta situación se presenta cuando un usuario inicia sesión en el sistema o cuando otro usuario consulta información básica de contacto del guía o chofer asignado a su ruta.

- **Campos del DTO:**

- **username** y **fullName** : Para identificar al usuario en la interfaz.
- **email** : Para propósitos de contacto.
- **age** : Información demográfica básica requerida por la empresa.
- **role** : Un campo que indique si es cliente, **DriverUser** o **TourGuideUser** .

- **¿Por qué es más adecuado?** Existen razones críticas de **seguridad y desacoplamiento**. La entidad **User** contiene el atributo **password**, el cual representa un riesgo de seguridad si se expone directamente en una respuesta de servicio hacia el cliente web. Además, las entidades de usuario pueden estar vinculadas a un gran historial de compras (**Purchase**), las cuales no son necesarias para una simple visualización de perfil o un proceso de login. El DTO actúa como una "copia desconectada" que protege la integridad de los datos sensibles y evita ciclos de serialización infinitos si existen relaciones bidireccionales en el modelo.

Inciso 34

¿Qué riesgos concretos tiene exponer entidades JPA directamente como respuesta de un servicio o endpoint? Mencionar al menos: ciclos de serialización y acoplamiento entre capas.

Respuesta:

Exponer las entidades JPA (objetos del modelo) directamente como respuesta en una API o servicio conlleva riesgos técnicos, de seguridad y de diseño que comprometen la estabilidad y el rendimiento de la aplicación.

- **Ciclos de Serialización:** Al convertir entidades en formatos como JSON para ser enviados al frontend, las **relaciones bidireccionales** (por ejemplo, una **Route** que conoce sus **Purchase** y una **Purchase** que conoce su **Route**) pueden generar bucles infinitos. El framework de serialización intentará recorrer las asociaciones de ida y vuelta constantemente, provocando errores en el servicio o respuestas excesivamente pesadas.
- **Acoplamiento entre Capas:** Exponer entidades acopla la interfaz de usuario directamente al **esquema físico de la base de datos**. Esto significa que cualquier cambio menor en la estructura de una tabla o en el mapeo de una entidad obligaría a modificar la lógica de presentación en el frontend, rompiendo la independencia entre capas que busca una arquitectura robusta.
- **Exposición de Lógica y Datos Sensibles:** Las entidades no son solo datos; contienen **lógica de negocio**. Si se devuelven directamente, se corre el riesgo de que el cliente (frontend) intente ejecutar métodos del modelo en forma indebida. Además, las entidades pueden contener atributos que no

deben ser públicos, como contraseñas, versiones internas de concurrencia o información histórica innecesaria para ese caso de uso específico.

- **Problemas de Performance y Lazy Loading:** Las entidades JPA suelen tener relaciones configuradas como **carga perezosa** (**lazy loading**). Si el servicio devuelve la entidad y el frontend intenta acceder a sus relaciones fuera del marco de la transacción, el sistema fallará lanzando una excepción (como `LazyInitializationException`). Si para evitar esto se carga todo el grafo de objetos a memoria, se genera un **tráfico de datos ineficiente** y se aumenta el consumo de recursos, impactando negativamente en la performance de las consultas.
- **Conflictos de Concurrencia:** Recuperar grafos de objetos demasiado grandes a memoria aumenta la probabilidad de que surjan conflictos de versiones si otros usuarios modifican esos mismos datos simultáneamente. Los **DTOs**, al ser copias desconectadas y volátiles, eliminan el riesgo de manipulación indebida de la instancia persistente fuera de la capa de servicios.

2. Implementación

Inciso 35

Definir e implementar un DTO que resuma información de las rutas del modelo: nombre de la ruta, cantidad de compras realizadas y el precio promedio de esas compras. Implementar la consulta correspondiente en `RouteRepository`.

V. Borrado Lógico

1. Conceptos y estrategias

Inciso 36

¿Que es el borrado lógico (soft delete)? Pensar en el modelo de tours: si un usuario decide darse de baja del sistema, ¿tiene sentido eliminar físicamente su registro? ¿Qué ocurriría con todas las compras que ese usuario ha realizado? Describir por qué el borrado lógico resuelve este problema y qué ventaja ofrece frente al borrado físico en este caso concreto.

Respuesta:

El **borrado lógico** (**soft delete**) es una estrategia de persistencia que consiste en marcar los registros de la base de datos como **inactivos** en lugar de eliminarlos físicamente del disco. En lugar de ejecutar una sentencia **DELETE**, se actualiza un atributo específico (como un booleano o una fecha) para indicar que el registro ya no debe ser considerado en las operaciones habituales de la aplicación.

Aplicación al modelo de tours

En el contexto de la aplicación del TP, **no tiene sentido eliminar físicamente el registro de un usuario** por las siguientes razones:

- **Integridad de los datos:** Un usuario está vinculado a un historial de compras (Purchase). En caso de eliminar físicamente al usuario, las filas de las compras quedarían huérfanas o, lo que es más probable, el motor de base de datos impediría la eliminación para no romper la integridad referencial (claves foráneas).
- **Pérdida de información histórica:** Para la empresa es vital mantener el registro de qué tours se vendieron y cuándo, incluso si el cliente ya no utiliza la aplicación. Eliminar físicamente al usuario borraría el rastro de esas transacciones comerciales, afectando estadísticas y reportes contables.*

Ventajas del borrado lógico en este caso

El borrado lógico resuelve estos problemas ofreciendo ventajas claras frente al borrado físico:

1. **Mantenimiento de la consistencia:** Permite que las entidades relacionadas (como las compras o las reseñas realizadas por el usuario) sigan siendo válidas y tengan una referencia al usuario que las originó, aunque este ya no pueda iniciar sesión.
2. **Facilidad de recuperación:** Si el usuario decide volver al sistema ("darse de alta" nuevamente), el proceso es tan simple como cambiar el estado del atributo de borrado.
3. **Auditoría y Análisis:** Permite que el sistema mantenga una "memoria" para análisis futuros o procesos legales donde sea necesario demostrar que un usuario existió y realizó ciertas acciones en una fecha determinada.
4. **Transparencia técnica:** Mediante herramientas como las anotaciones **@SQLDelete** (para interceptar la eliminación) y **@Where** (para filtrar

automáticamente los inactivos), el resto de la aplicación puede funcionar como si el registro realmente no existiera, evitando errores de lógica sin perder los datos físicamente.

Inciso 37

Describir dos estrategias para implementar soft delete en JPA/Hibernate: campo booleano (active/deleted) y campo de fecha (deletedAt). ¿Qué diferencias hay entre ambas en cuanto a consultas y recuperación de datos?

Respuesta:

Estrategia de Campo Booleano (`active` / `deleted`)

Esta es la forma más simple de implementar el borrado lógico. Consiste en agregar una columna de tipo booleano a la tabla (por ejemplo, `is_deleted` o `active`).

- **Funcionamiento:** Cuando se solicita borrar un registro, se intercepta la operación mediante `@SQLDelete` para ejecutar un `UPDATE` que cambie el valor de `false` a `true`.
- **Filtro:** Se utiliza la anotación `@Where(clause = "is_deleted = false")` sobre la entidad para que Hibernate añada automáticamente esa condición a todas las consultas `SELECT` (`findAll`, `findById`, etc.).

Estrategia de Campo de Fecha (`deletedAt`)

Esta estrategia utiliza un atributo de tipo fecha/hora (ej. `LocalDateTime` o `Date`) en lugar de un simple sí/no.

- **Funcionamiento:** Un registro se considera "activo" si el campo es `null`. Al realizar el borrado lógico, se guarda la **fecha y hora exacta** del sistema en dicha columna.
- **Filtro:** Las consultas se filtran automáticamente mediante la cláusula `@Where(clause = "deleted_at IS NULL")`.

En ambos casos la recuperación es sumamente sencilla: basta con realizar un `UPDATE` manual o mediante una consulta nativa para resetear el valor (volver a `false` o a `null`).

La estrategia de fecha (`deletedAt`) es superior para el análisis de datos, ya que responde no solo si un registro fue borrado, sino **cuándo ocurrió**. La estrategia booleana requiere una columna de auditoría extra si se necesita esa información.

La estrategia booleana es ligeramente más sencilla de filtrar (`= false` vs `IS NULL`), aunque en términos de rendimiento en motores de bases de datos modernos la diferencia es despreciable.

Inciso 38

¿Que es la anotación `@SQLDelete` de JPA? ¿Qué permite hacer? ¿Cómo se combina con `@Where` para que las consultas ordinarias ignoren los registros borrados? Mostrar un ejemplo de uso sobre la entidad `User`.

Respuesta:

La anotación `@SQLDelete` es una herramienta de Hibernate (utilizada frecuentemente en proyectos JPA) que permite **sobrescribir la sentencia SQL predeterminada** que el framework genera para eliminar una entidad.

En lugar de permitir que Hibernate ejecute un borrado físico (`DELETE FROM ...`), esta anotación permite interceptar la operación y ejecutar en su lugar un `UPDATE` para realizar un **borrado lógico (soft delete)**. Su propósito principal es cambiar el estado de un registro (por ejemplo, de "activo" a "inactivo") sin removerlo físicamente del disco.

Combinación con el where

1. **@SQLDelete**: Redefine la acción de borrar para que, al invocar `repository.delete(user)` , se ejecute una actualización del campo de control (ej. `is_deleted = true`).
2. **@Where**: Define una cláusula SQL que Hibernate **añadirá automáticamente** a todas las consultas ordinarias (`SELECT`) que involucren a esa entidad (como `findAll` , `findById` o Query Methods).
3. **Resultado**: De esta manera, el sistema "cree" que los registros marcados como borrados ya no existen, ya que son filtrados globalmente en cada consulta de lectura.

Ejemplo:

```

@Entity
@Table(name = "ITEM_USER")
@SQLDelete(sql = "UPDATE ITEM_USER SET is_deleted = true WHERE
oid_user = ?")
@Where(clause = "is_deleted = false")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;
    // 3. Atributo para controlar el borrado lógico
    @Column(name = "is_deleted")
    private boolean isDeleted = false;

    // Getters y Setters
}

```

Inciso 39

Implementar soft delete para la entidad User usando @SQLDelete y @Where. La implementación debe:

1. Agregar el campo necesario a la entidad User (campo y anotaciones).
2. Redefinir el comportamiento de delete para que marque el registro en lugar de eliminarlo.
3. Asegurar que todas las consultas ordinarias (findAll, findById, Query Methods) excluyan automáticamente los usuarios borrados.

Inciso 40

La implementación actual muestra el método canBeDeactivate() en User, DriverUser y TourGuideUser. El objetivo de estos métodos es evitar que se eliminen usuarios que poseen compras o asignaciones y, por tanto queden inconsistencias en la base de datos.

Describe como esta nueva implementación afecta dichos métodos.

Respuesta:

La nueva implementación del **borrado lógico (soft delete)** mediante `@SQLDelete` y `@Where` afecta significativamente a los métodos `canBeDeactivate()`, transformando su rol dentro de la arquitectura de la siguiente manera:

- **Pérdida de necesidad técnica para la integridad:** El propósito original de `canBeDeactivate()` era actuar como una salvaguarda para evitar errores de integridad referencial (claves foráneas). Al usar borrado lógico, el registro **permanece físicamente en la base de datos**, por lo que las restricciones de integridad no se ven amenazadas y el riesgo de inconsistencia técnica desaparece.
- **Redundancia como validador de borrado:** Con la anotación `@SQLDelete`, cualquier llamada a `repository.delete(user)` se traduce automáticamente en un `UPDATE` del campo de control (ej. `is_deleted = true`). Esto significa que el framework garantiza la permanencia del dato independientemente de si `canBeDeactivate()` permite o no la operación, haciendo que el método pierda su peso como guardián de la base de datos.
- **Cambio de lógica de negocio a validación de estado:** Aunque ya no sea necesario para prevenir errores de SQL, el método podría sobrevivir si la empresa tiene una **regla de negocio** que prohíba que un usuario sea "desactivado" mientras tenga tareas pendientes (por ejemplo, un guía con un tour mañana). Sin embargo, su función ya no sería evitar una "inconsistencia en la base de datos", sino validar una **transición de estado permitida** desde el punto de vista del negocio.
- **Transparencia en las relaciones:** Dado que Hibernate ahora filtra automáticamente los registros borrados mediante `@Where`, el sistema puede seguir navegando las relaciones de forma segura. Las compras antiguas seguirán apuntando al usuario (que físicamente existe), pero dicho usuario no aparecerá en los listados activos de la aplicación.